

bok25

**基
礎**

課、操作系統



操作系统的运行环境



OS的运行环境

OS引导

OS内核

CPU的双重工作模式

特权指令&非特权指令

系统调用

中断和异常的处理



OS引导

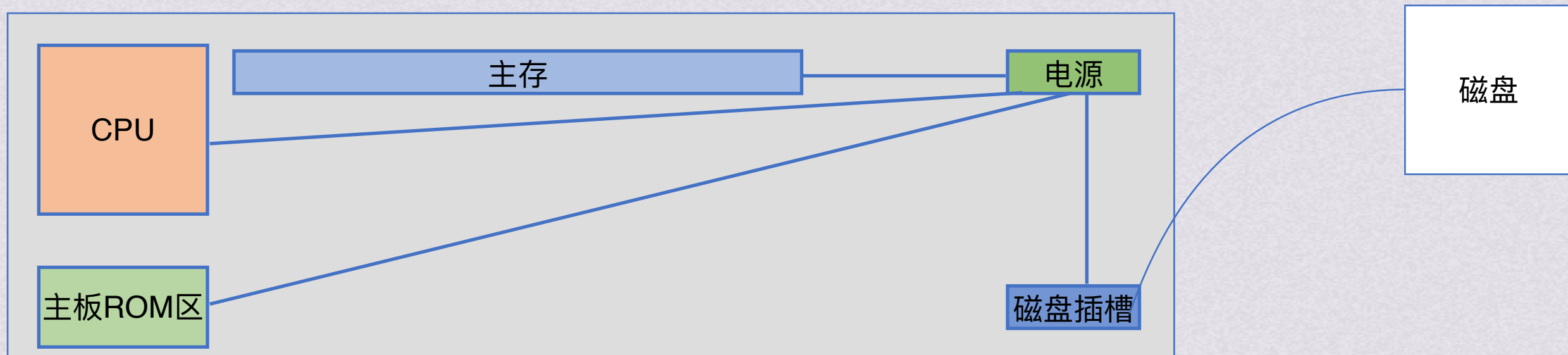
- OS内核
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用
- 中断和异常的处理



OS引导

从启动电源到进入操作系统的整个过程

1. 计算机启动时，首先进行电源自检，确保所有硬件设备正常并准备就绪



电脑主板

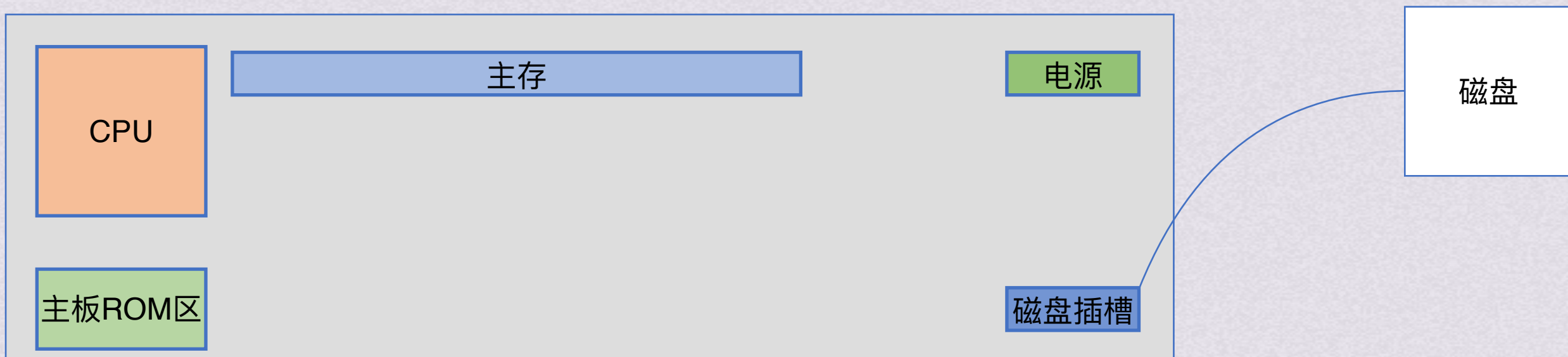
OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用
中断和异常的处理



OS引导

从启动电源到进入操作系统的整个过程

1. 计算机启动时，首先进行电源自检，确保所有硬件设备正常并准备就绪
2. 计算机主板ROM区BIOS程序被激活，负责初始化硬件，进一步自检



电脑主板

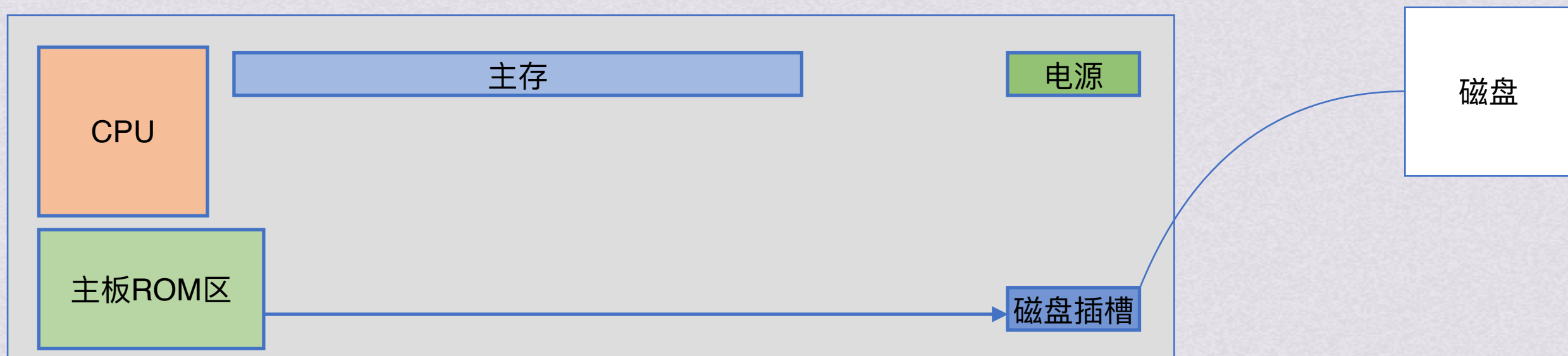
OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用
中断和异常的处理



OS引导

从启动电源到进入操作系统的整个过程

1. 计算机启动时，首先进行电源自检，确保所有硬件设备正常并准备就绪
2. 计算机主板ROM区BIOS程序被激活，负责初始化硬件，进一步自检
3. BIOS读取硬盘0号扇区的主引导记录MBR，MBR包括磁盘引导程序和分区表信息



OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用
中断和异常的处理

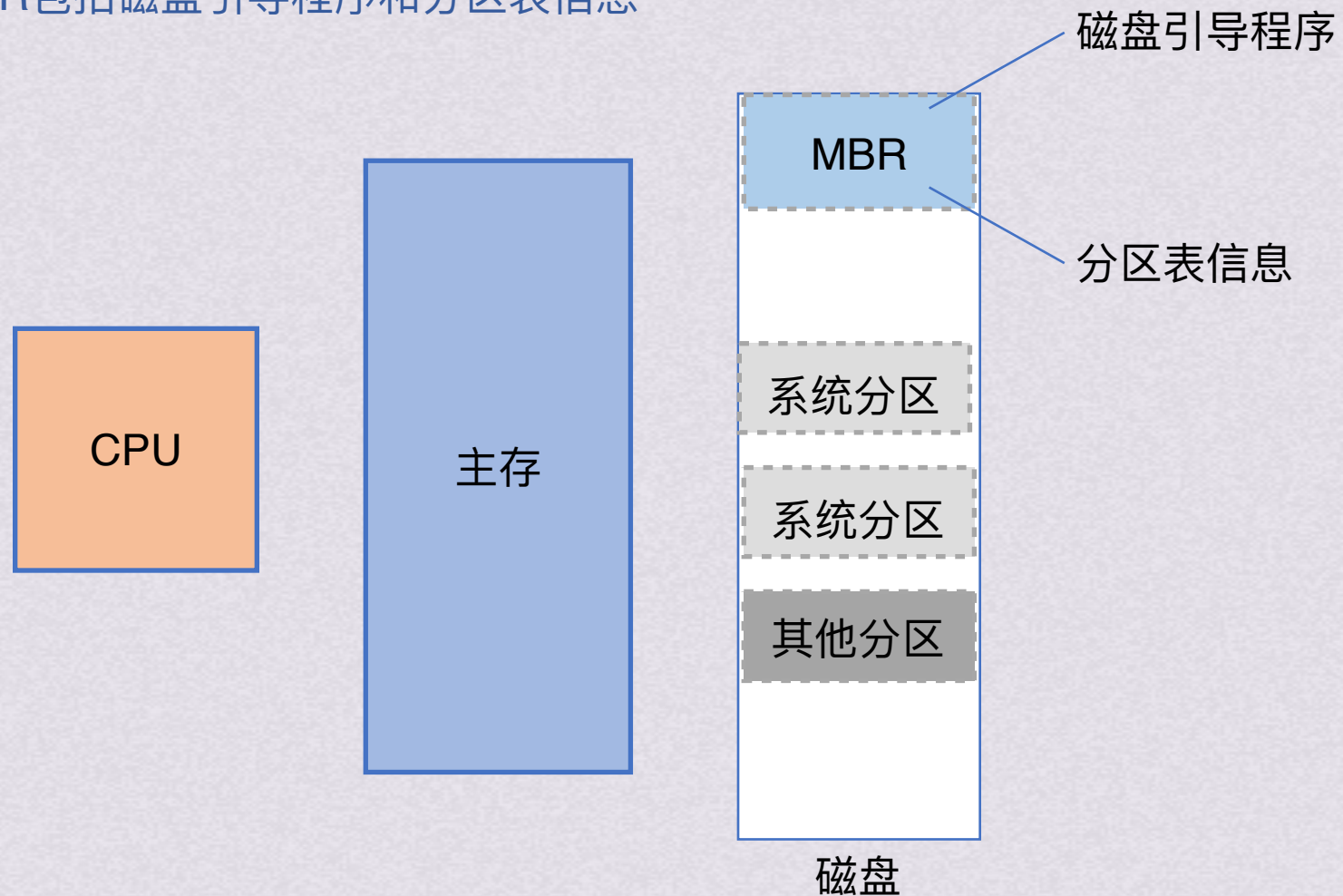
电脑主板



OS引导

从启动电源到进入操作系统的整个过程

1. 计算机启动时，首先进行电源自检，确保所有硬件设备正常并准备就绪
2. 计算机主板ROM区BIOS程序被激活，负责初始化硬件，进一步自检
3. BIOS读取硬盘0号扇区的主引导记录MBR，MBR包括磁盘引导程序和分区表信息



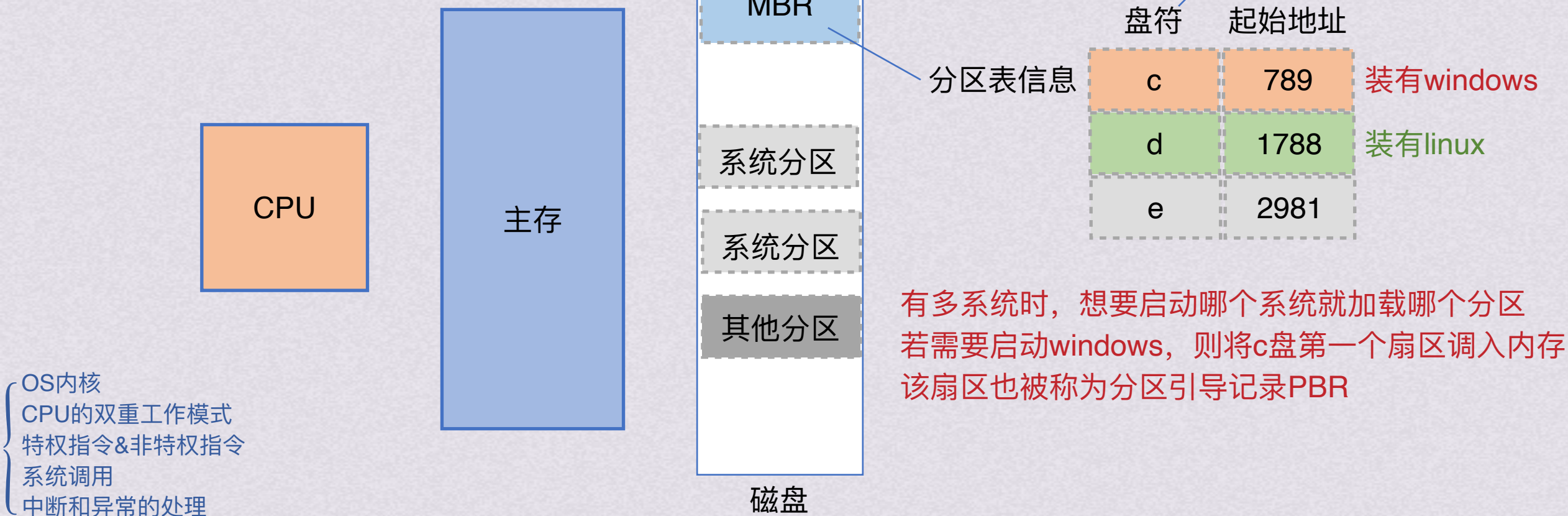
OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用
中断和异常的处理



OS引导

从启动电源到进入操作系统的整个过程

1. 计算机启动时，首先进行电源自检，确保所有硬件设备正常并准备就绪
2. 计算机主板ROM区BIOS程序被激活，负责初始化硬件，进一步自检
3. BIOS读取硬盘0号扇区的主引导记录MBR，MBR包括磁盘引导程序和分区表信息
4. 从分区表可得知操作系统所在系统分区位置，系统分区的0号扇区存放分区引导记录PBR，调入该扇区内的操作引导程序进入主存得知OS内核位置并将其加载入主存



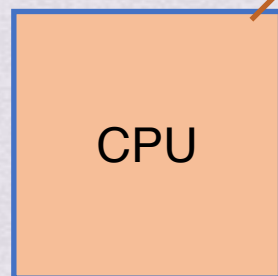


OS引导

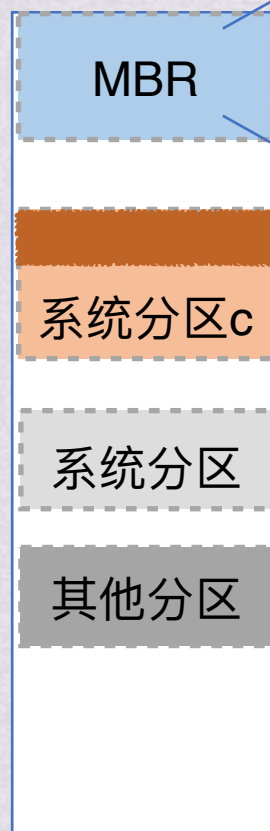
从启动电源到进入操作系统的整个过程

1. 计算机启动时，首先进行电源自检，确保所有硬件设备正常并准备就绪
2. 计算机主板ROM区BIOS程序被激活，负责初始化硬件，进一步自检
3. BIOS读取硬盘0号扇区的主引导记录MBR，MBR包括磁盘引导程序和分区表信息
4. 从分区表可得知操作系统所在系统分区位置，系统分区的0号扇区存放分区引导记录PBR，调入该扇区内的操作引导程序进入主存得知OS内核位置并将其加载入主存

CPU执行这一部分代码，
windows成功被启动



主存



MBR

磁盘引导程序

分区表信息

系统分区c

系统分区

其他分区

磁盘

(分区标识)

盘符

起始地址

c	789
d	1788
e	2981

装有windows

装有linux

有多系统时，想要启动哪个系统就加载哪个分区
若需要启动windows，则将c盘第一个扇区调入内存
该扇区也被称为分区引导记录PBR

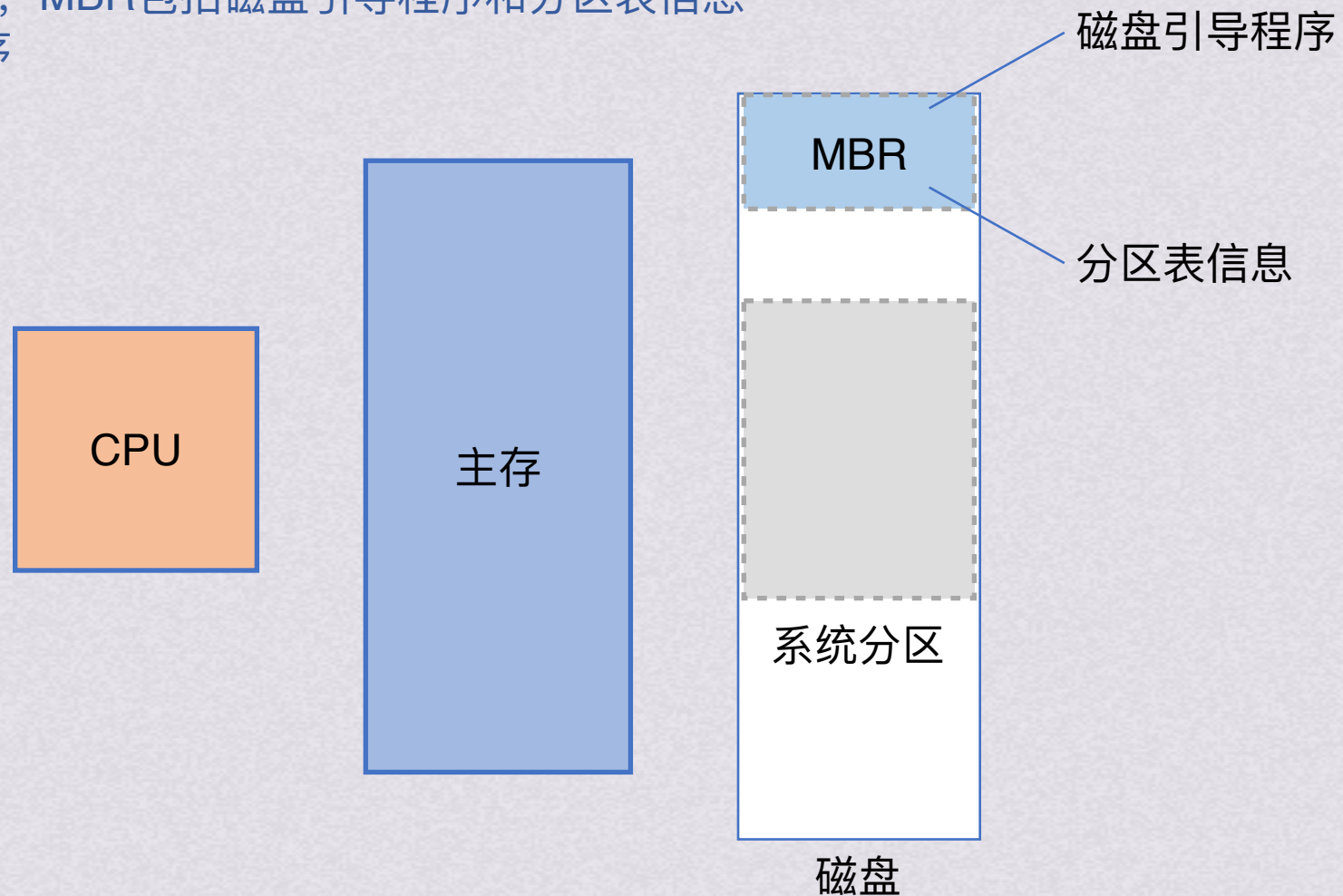
OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用
中断和异常的处理



OS引导

从启动电源到进入操作系统的整个过程

1. 计算机启动时，首先进行电源自检，确保所有硬件设备正常并准备就绪
2. 计算机主板ROM区BIOS程序被激活，负责初始化硬件，进一步自检
3. BIOS从硬盘的主引导记录MBR中读取引导程序，MBR包括磁盘引导程序和分区表信息
4. 利用引导程序找到系统分区，加载启动管理程序



OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用
中断和异常的处理

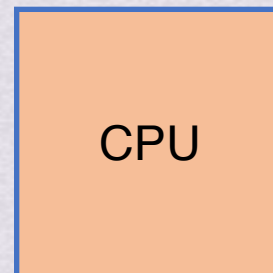


OS引导

从启动电源到进入操作系统的整个过程

1. 计算机启动时，首先进行电源自检，确保所有硬件设备正常并准备就绪
2. 计算机主板ROM区BIOS程序被激活，负责初始化硬件，进一步自检
3. BIOS从硬盘的主引导记录MBR中读取引导程序，MBR包括磁盘引导程序和分区表信息
4. 利用引导程序找到系统分区，加载启动管理程序
5. 启动管理程序加载OS引导加载程序
6. 操作系统内核被加载到内存，开始执行初始化程序

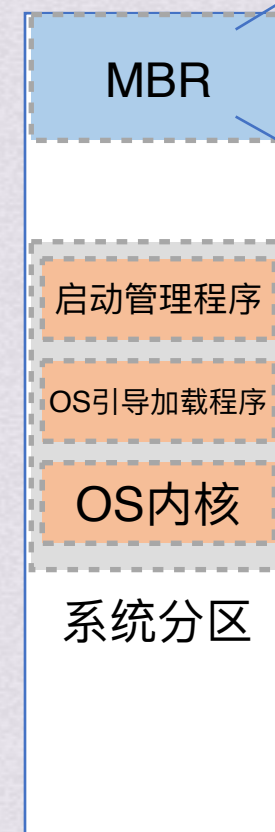
初始化完成后，系统变为可用状态



CPU



主存



MBR

启动管理程序

OS引导加载程序

OS内核

系统分区

磁盘

磁盘引导程序

分区表信息

OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用
中断和异常的处理



OS引导

{ OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用
中断和异常的处理



OS内核

OS引导

CPU的双重工作模式

特权指令&非特权指令

系统调用

中断和异常的处理



OS内核

OS中与硬件紧密相关的模块，负责管理系统的各种资源

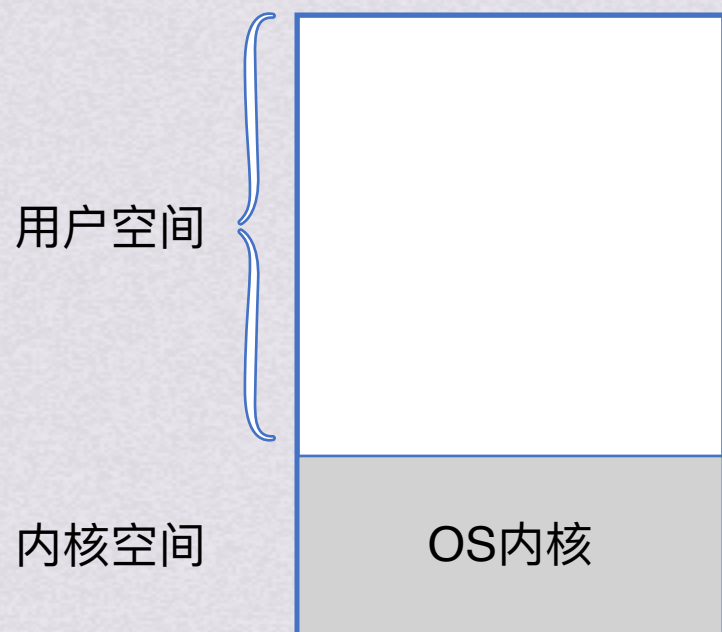


- OS引导
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用
- 中断和异常的处理

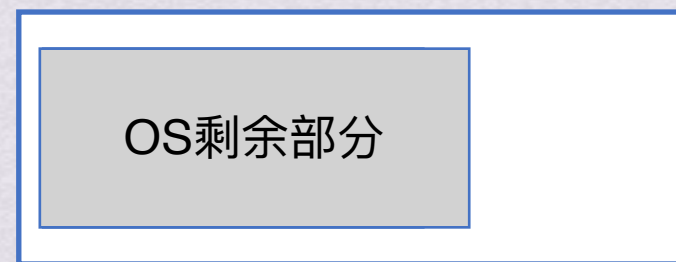


OS内核

OS中与硬件紧密相关的模块，负责管理系统的各种资源



内存地址空间



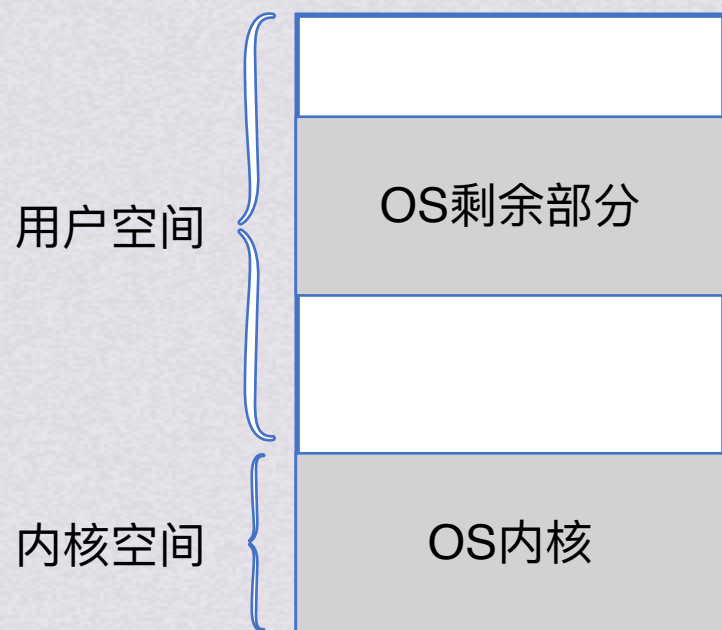
磁盘

- OS引导
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用
- 中断和异常的处理

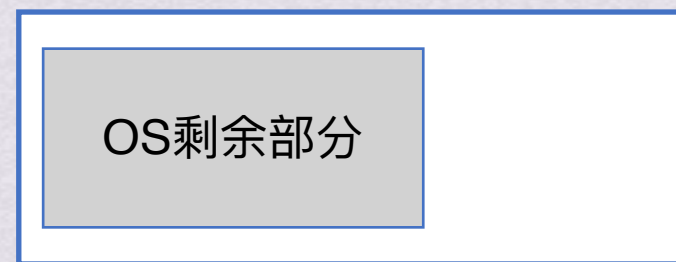


OS内核

OS中与硬件紧密相关的模块，负责管理系统的各种资源



内存地址空间



磁盘

- OS引导
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用
- 中断和异常的处理



OS内核

OS中与硬件紧密相关的模块，负责管理系统的各种资源

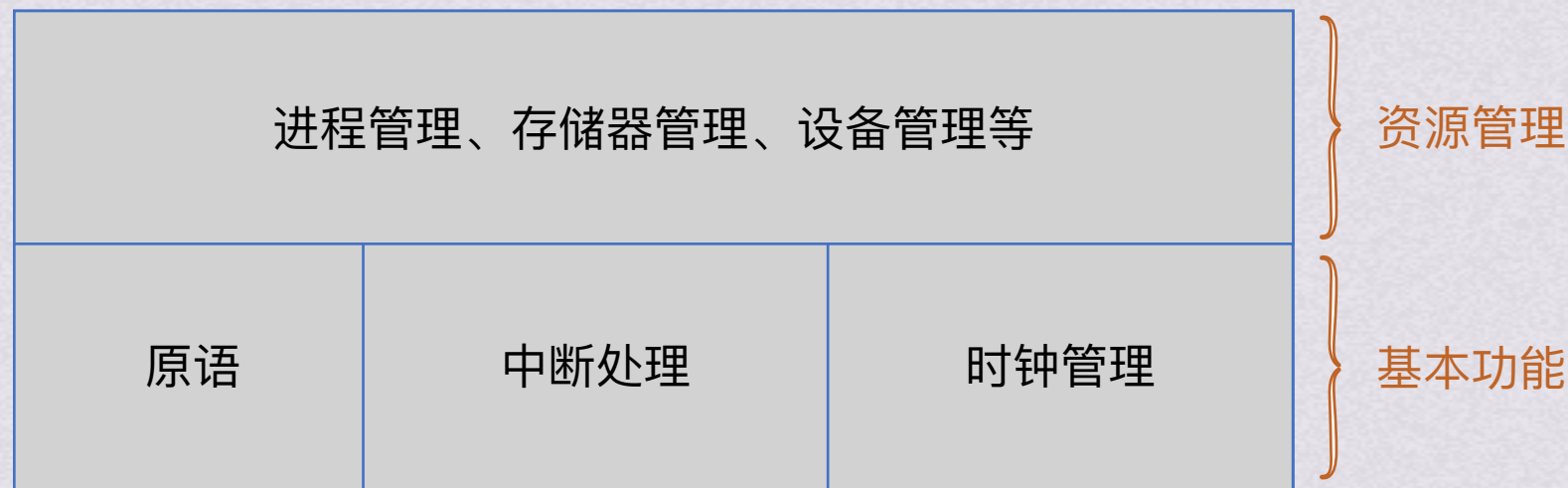


OS引导
CPU的双重工作模式
特权指令&非特权指令
系统调用
中断和异常的处理



OS内核

OS中与硬件紧密相关的模块，负责管理系统的各种资源

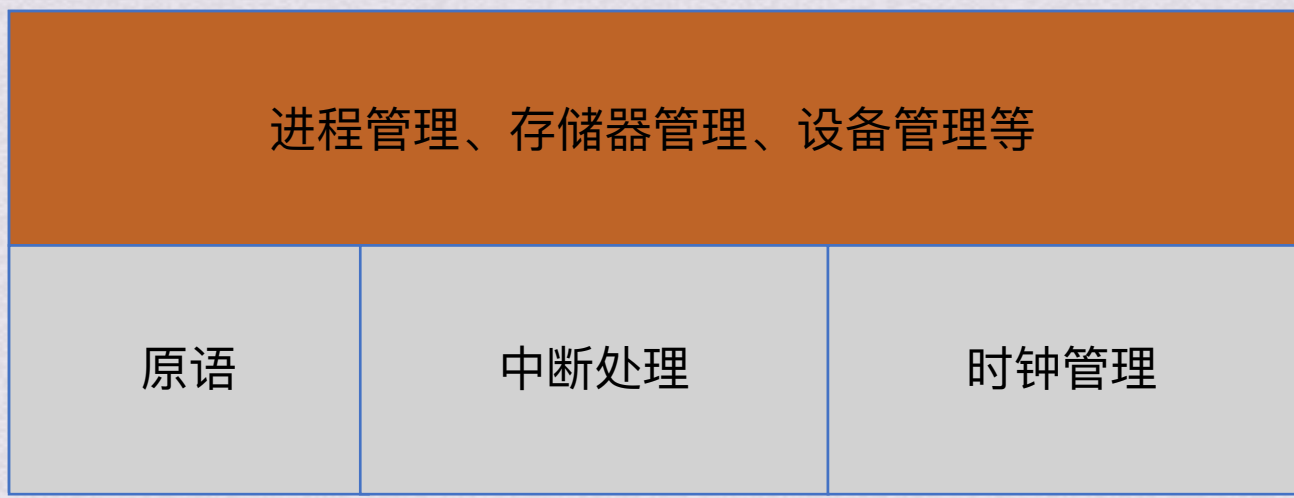


OS内核

- OS引导
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用
- 中断和异常的处理

OS内核

OS中与硬件紧密相关的模块，负责管理系统的各种资源



OS内核

进程管理

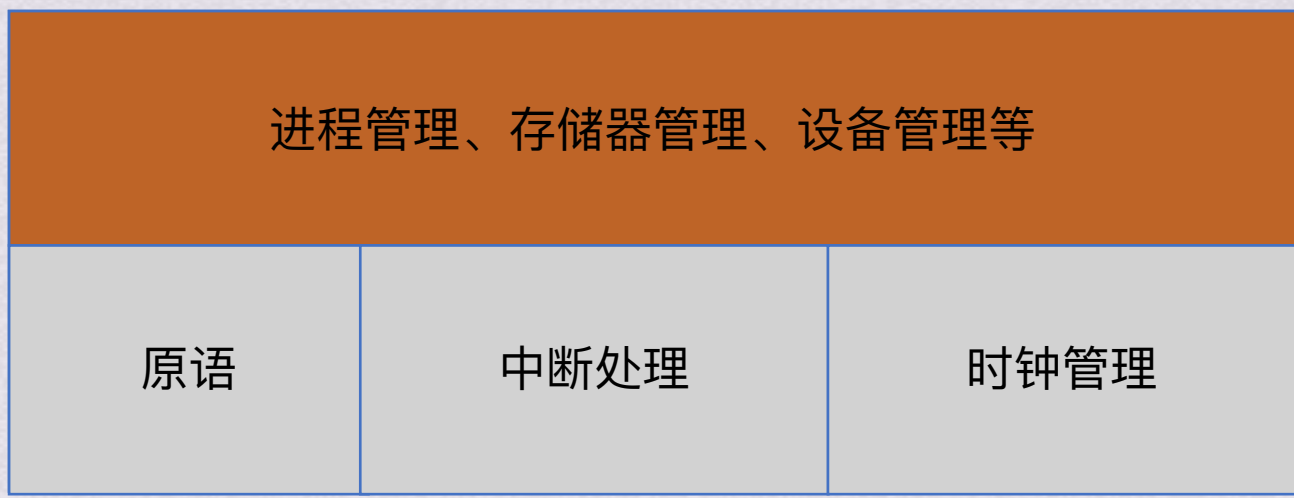
确保OS可以高效、公平、安全地同时运行多个应用程序

*将在第二章进一步学习

- OS引导
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用
- 中断和异常的处理

OS内核

OS中与硬件紧密相关的模块，负责管理系统的各种资源



OS内核

存储器管理

确保有效、安全地分配管理计算机系统中的内存资源

*将在第三章进一步学习

- OS引导
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用
- 中断和异常的处理



OS内核

OS中与硬件紧密相关的模块，负责管理系统的各种资源



进程管理、存储器管理、设备管理等

原语

中断处理

时钟管理

OS内核

设备管理

确保所有外围设备和硬件都能有效
且安全地与计算机系统交互

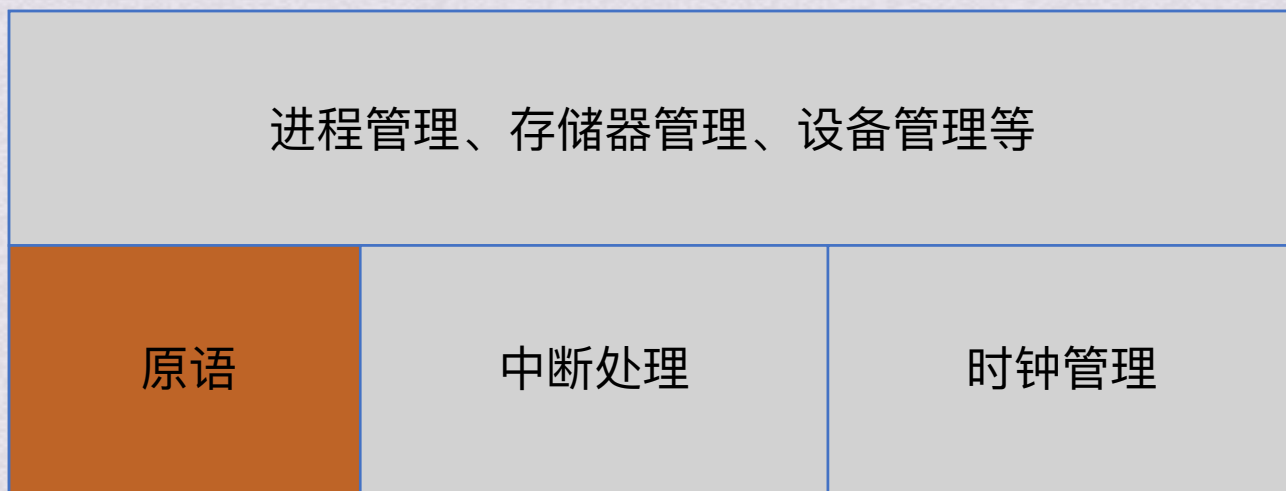
*将在第五章进一步学习

OS引导
CPU的双重工作模式
特权指令&非特权指令
系统调用
中断和异常的处理



OS内核

OS中与硬件紧密相关的模块，负责管理系统的各种资源



OS内核

- OS引导
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用
- 中断和异常的处理

原语

在单个步骤中完成OS的基础构件

处于操作系统最低层，运行时不可以被打断

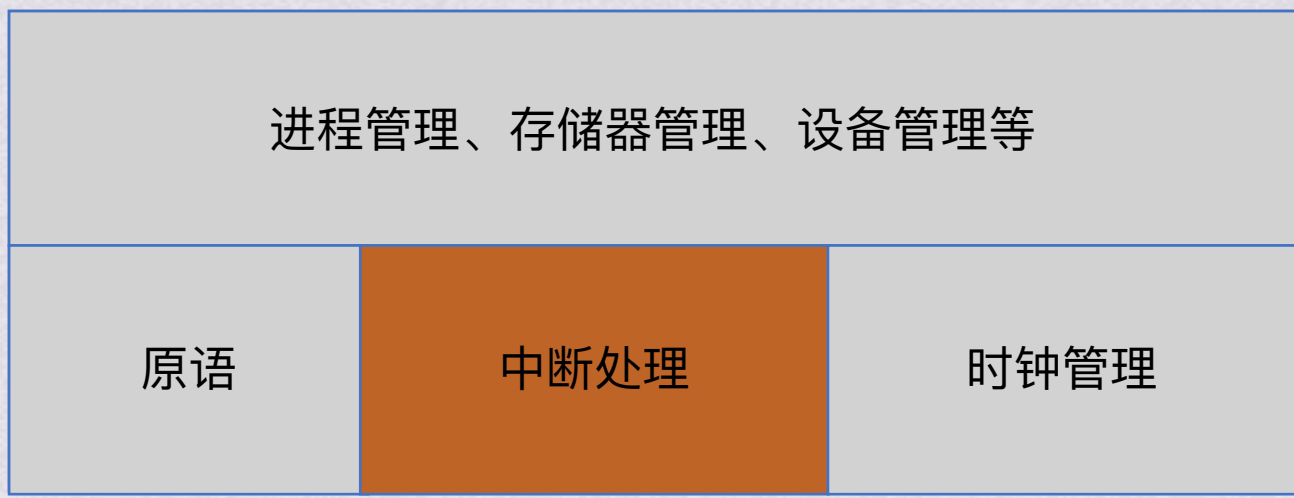
运行时长较短，调用频繁

linux中的常见原语

- fork()
- exec()
- wait()
- open()
- ...

OS内核

OS中与硬件紧密相关的模块，负责管理系统的各种资源



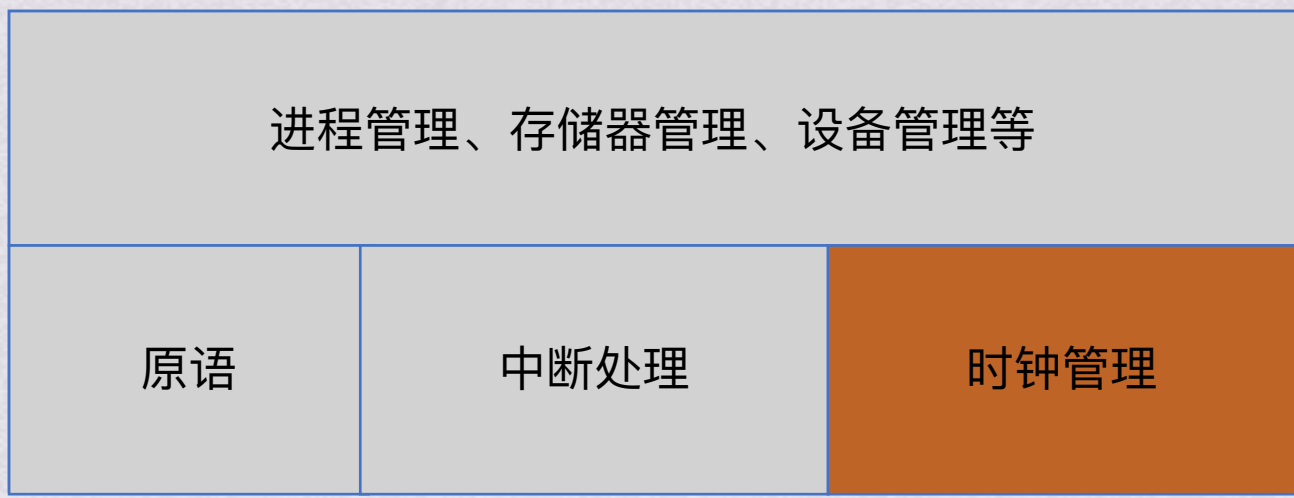
OS内核

中断处理

系统调用、键盘命令输入、进程调度、设备驱动等很多重要的活动都依赖于中断

- OS引导
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用
- 中断和异常的处理

OS内核 OS中与硬件紧密相关的模块，负责管理系统的各种资源



OS内核

时钟管理

OS用来处理和控制时间和时间相关函数的机制

- OS引导
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用
- 中断和异常的处理

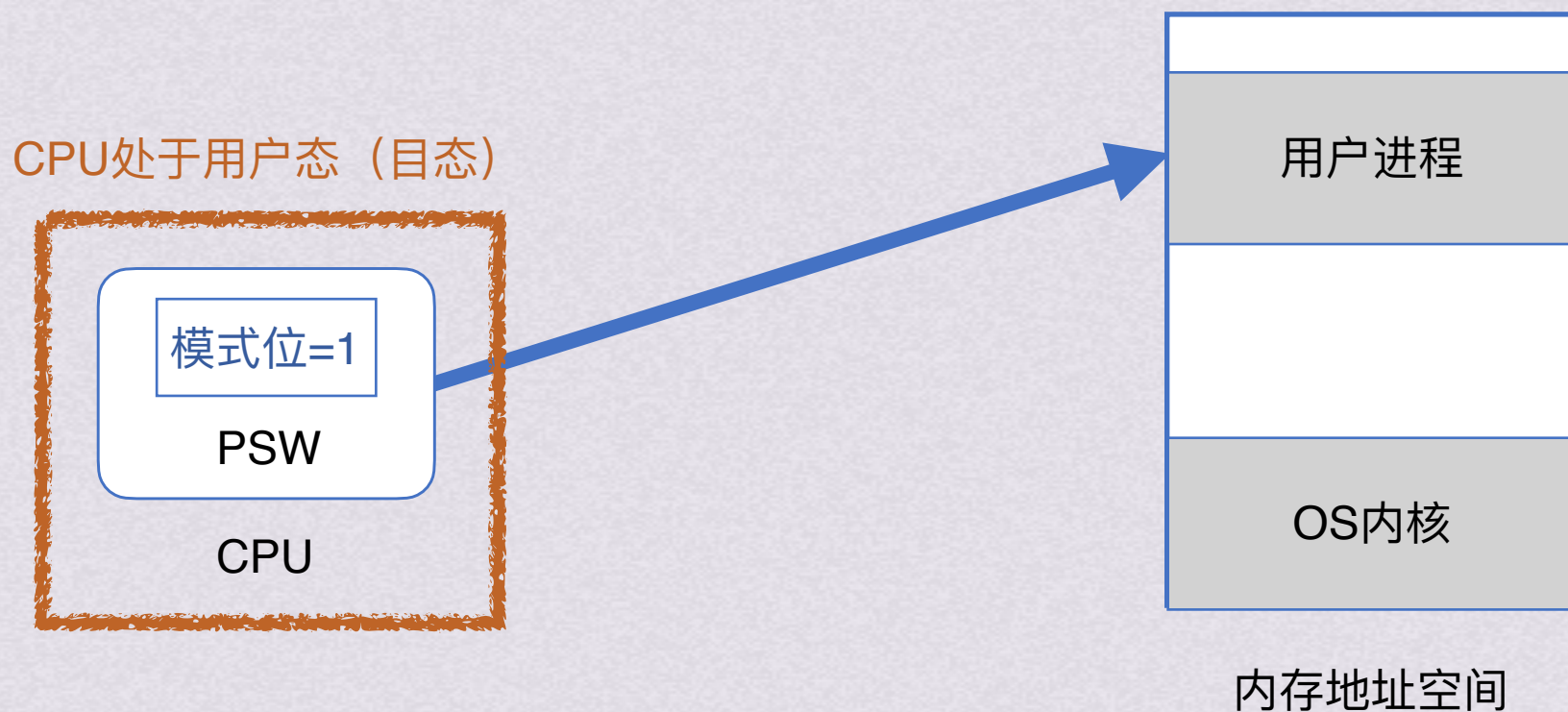


CPU的双重工作模式

{ OS引导
OS内核
特权指令&非特权指令
系统调用
中断和异常的处理



CPU的双重工作模式

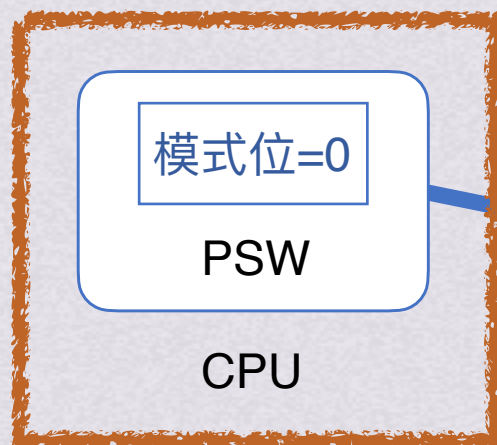


OS引导
OS内核
特权指令&非特权指令
系统调用
中断和异常的处理



CPU的双重工作模式

CPU处于内核态（管态）

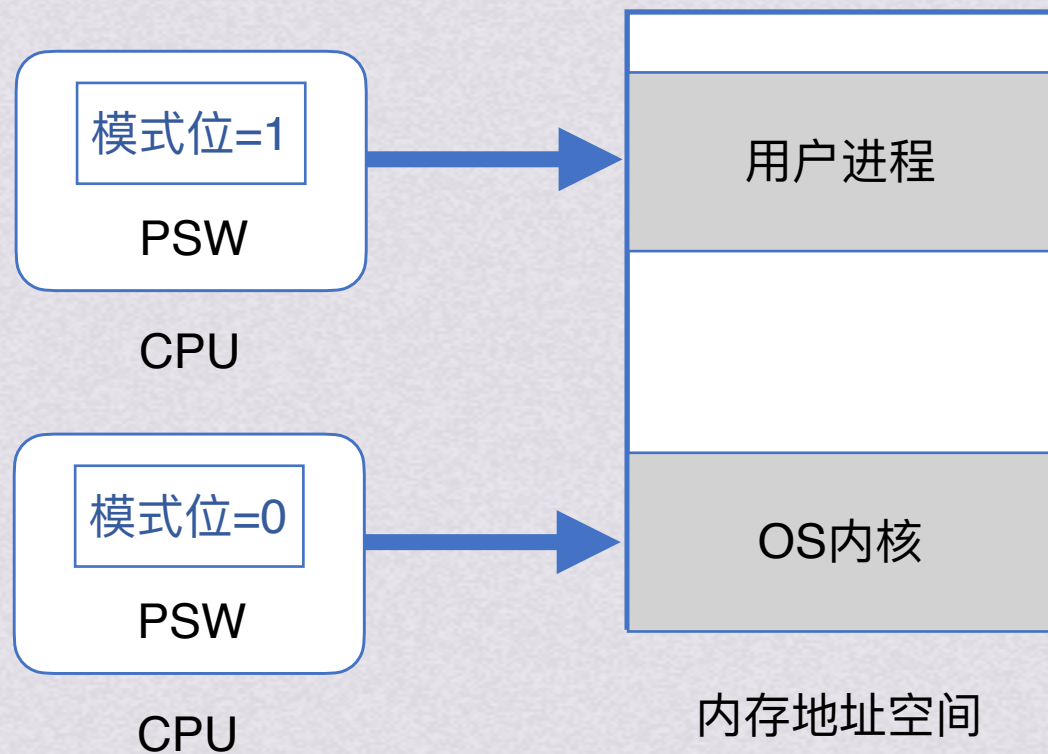


内存地址空间

OS引导
OS内核
特权指令&非特权指令
系统调用
中断和异常的处理



CPU的双重工作模式



CPU在用户态可以访问内存的用户空间

CPU只有在内核态可以访问内存的内核空间
(其实就是OS内核)

OS引导
OS内核
特权指令&非特权指令
系统调用
中断和异常的处理

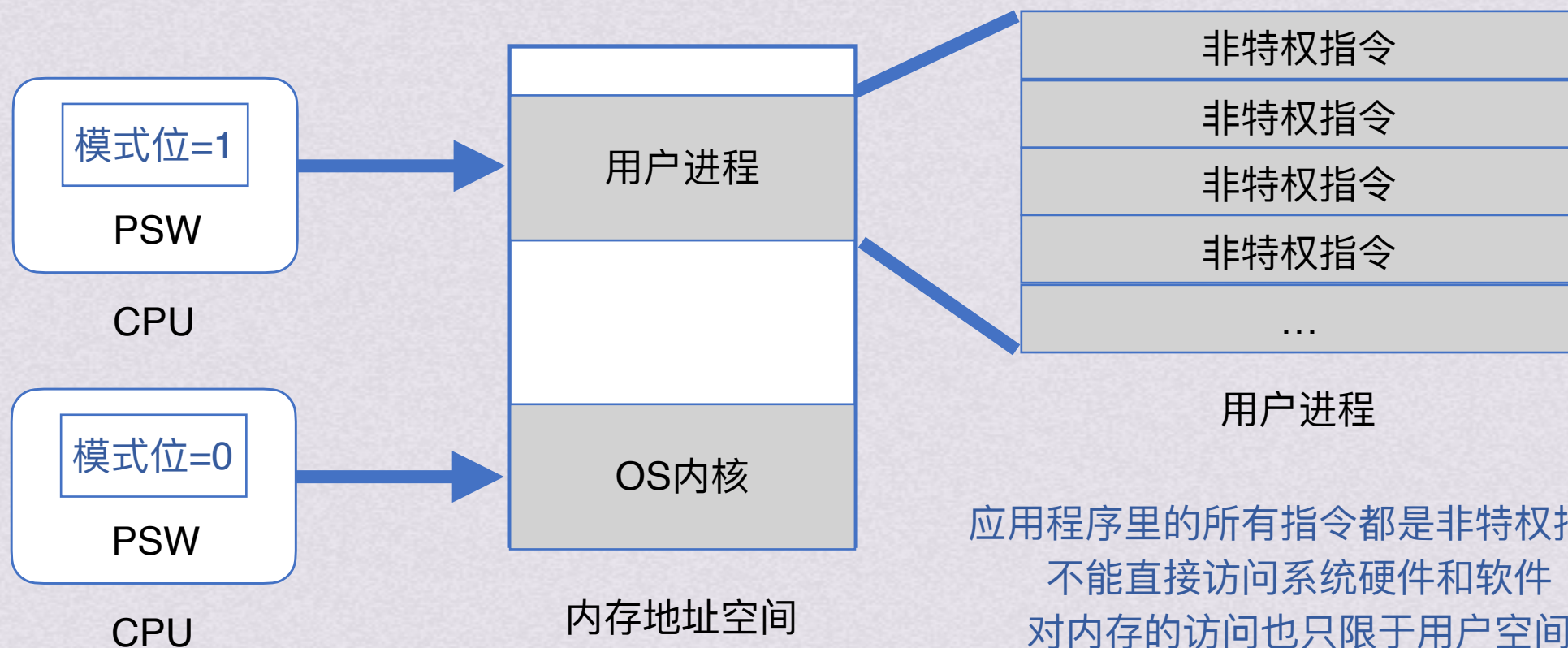


特权指令&非特权指令

{ OS引导
OS内核
CPU的双重工作模式
系统调用
中断和异常的处理



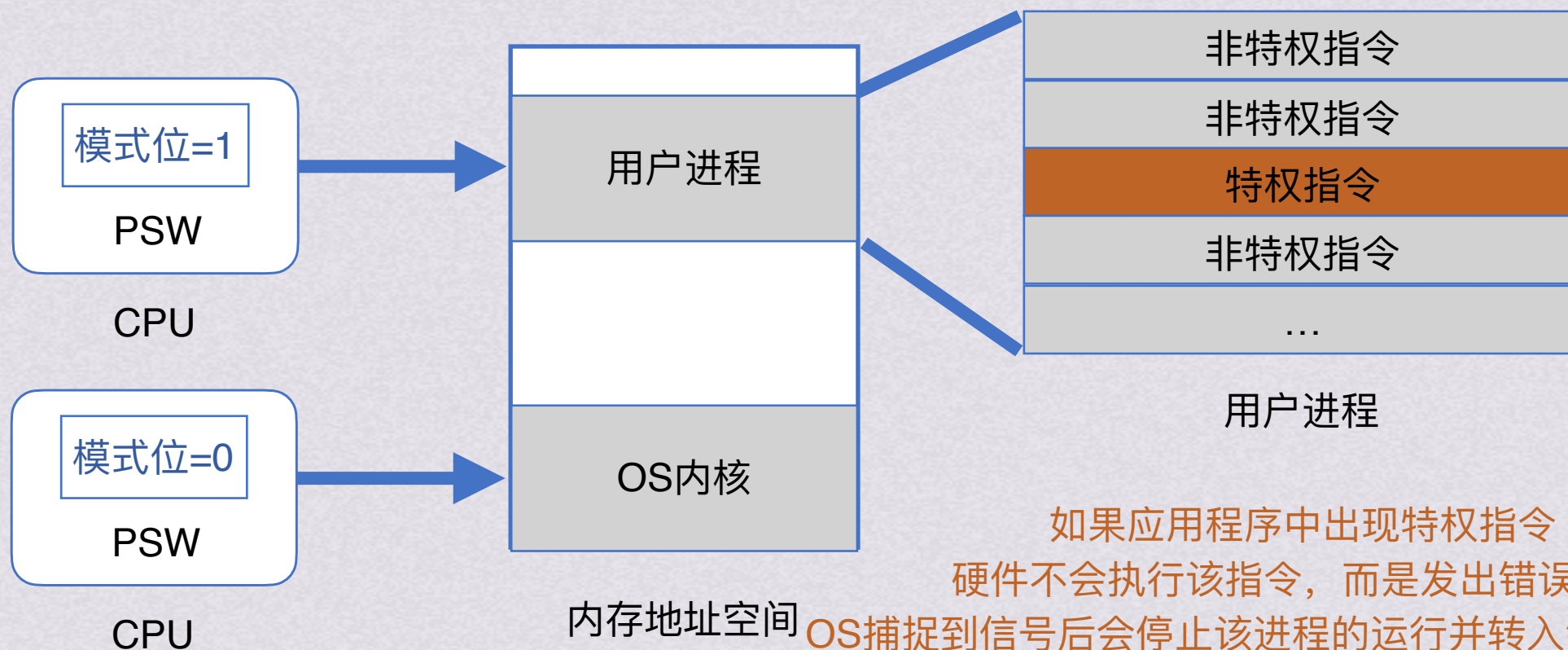
特权指令&非特权指令



OS引导
OS内核
CPU的双重工作模式
系统调用
中断和异常的处理



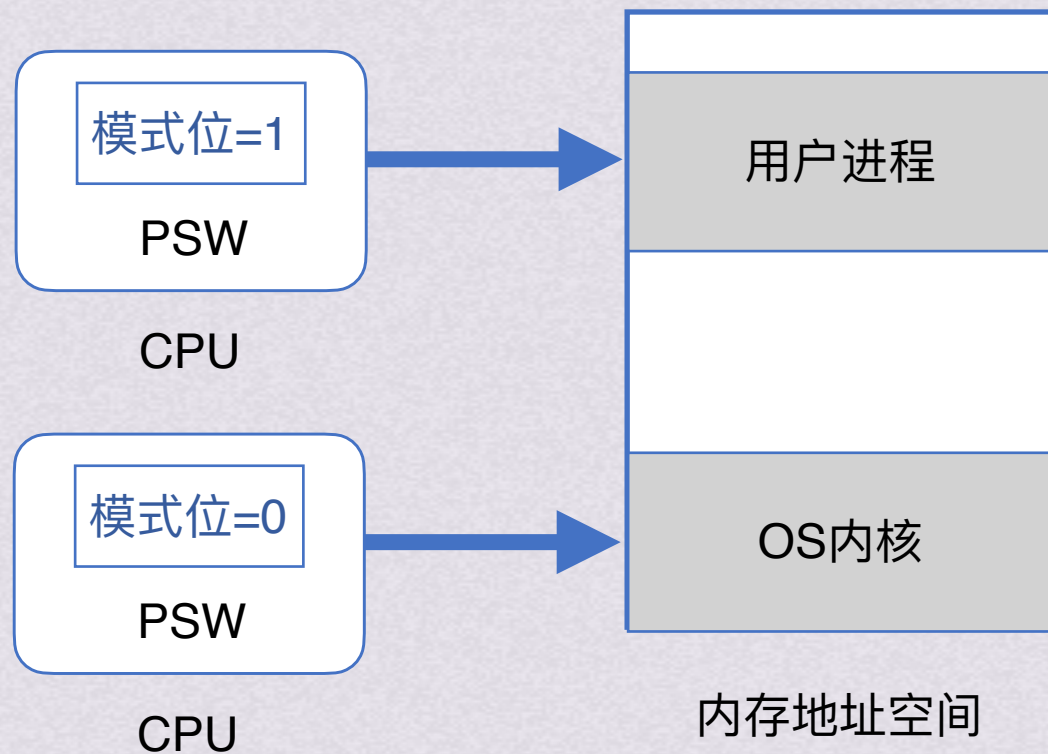
特权指令 & 非特权指令



OS引导
OS内核
CPU的双重工作模式
系统调用
中断和异常的处理



特权指令&非特权指令



特权指令

特权指令是CPU在内核态才能运行的指令
它对内存的访问范围基本不受限制

OS引导
OS内核
CPU的双重工作模式
系统调用
中断和异常的处理

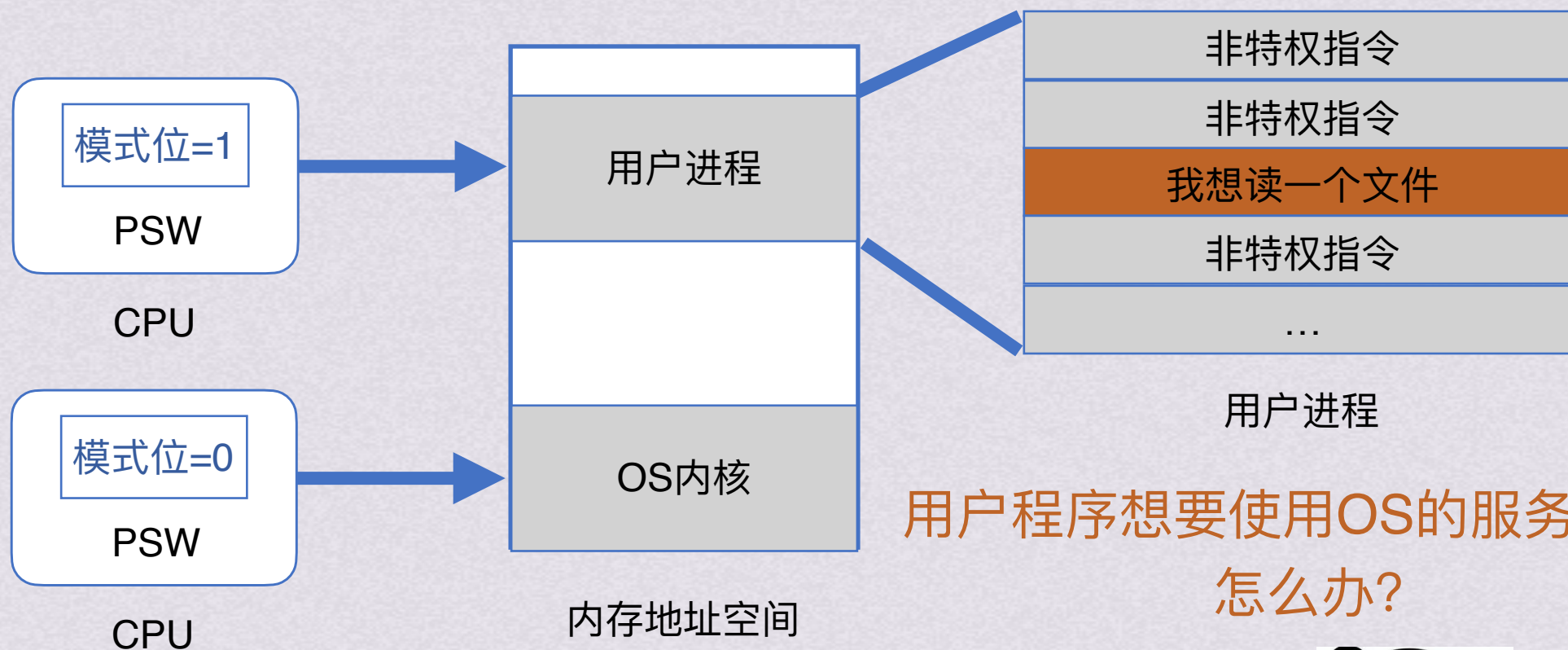


系统调用

OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理

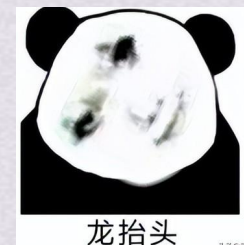


系统调用



用户程序想要使用OS的服务的时候
怎么办？

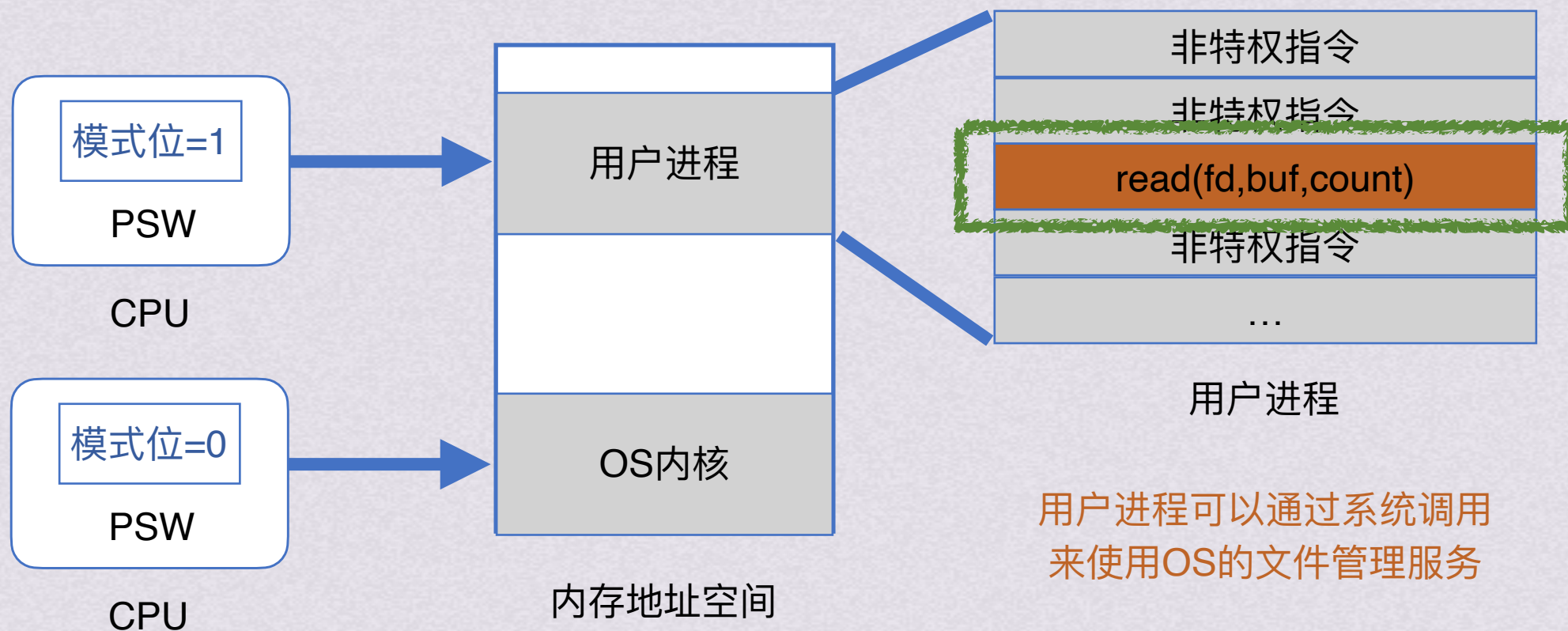
内核：



OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理



系统调用

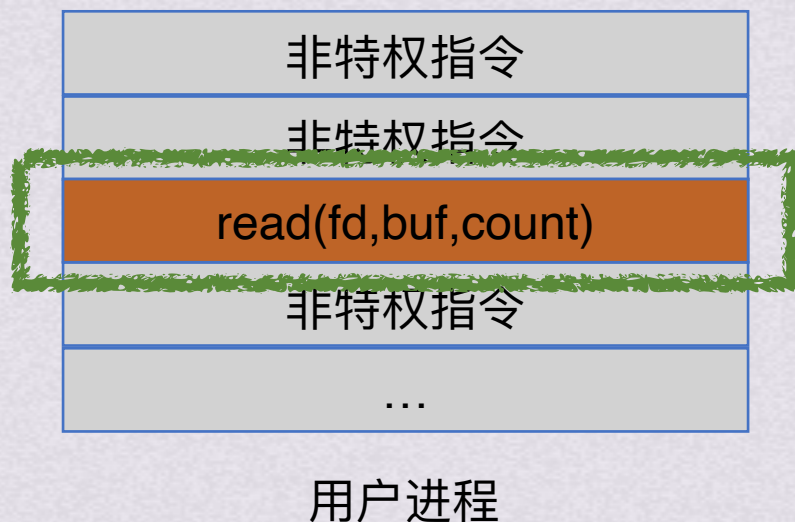


OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理



系统调用

系统调用 (system call) 形如一组打包好的库函数
这些函数的功能需要由OS内核来实现
系统调用提供了用户程序和OS内核之间的接口
应用程序通过系统调用来获得OS的服务



1. 文件操作:

- **open()**: 打开一个文件。
- **read()**: 从文件中读取数据。
- **write()**: 向文件写入数据。
- **close()**: 关闭一个打开的文件。
- **lseek()**: 移动文件的读写位置。
- **unlink()**: 删除一个文件的目的文件。
- **stat(), fstat()**: 获取文件的状态。

2. 进程控制:

- **fork()**: 复制当前进程，创建一个新的进程。
- **exit()**: 结束当前进程。
- **execve()**: 执行一个新程序，替换当前进程的映像。
- **wait(), waitpid()**: 等待子进程结束，并获取子进程的终止状态。

3. 进程间通信:

- **pipe()**: 创建一个管道，用于进程间的通信。
- **shmget(), shmat(), shmctl()**: 进行共享内存的操作。
- **semget(), semop(), semctl()**: 信号量的操作。
- **msgget(), msgsnd(), msgrcv()**: 消息队列的操作。

OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理



系统调用

系统调用的过程

用户代码中的read()只是一个库函数
并非真正的系统调用

库函数准备所有必要的参数
设置寄存器

这个指令不会显式地出现在用户代码中
用户会以为是自己的read()引发了系统调用
库函数偷偷地执行一个**软中断或陷阱指令**
发起真正的系统调用

read(fd,buf,count)

int 0x80/syscall

模式位=1

PSW

CPU

OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理



系统调用

系统调用的过程

用户代码中的read()只是一个库函数
并非真正的系统调用

库函数准备所有必要的参数
设置寄存器

read(fd,buf,count)

int 0x80/syscall

模式位=0

PSW

CPU

这个指令不会显式地出现在用户代码中
用户会以为是自己的read()引发了系统调用
库函数偷偷地执行一个软中断或陷阱指令
发起真正的系统调用

执行软中断/陷阱指令
可以引发自陷异常
导致CPU从用户态陷入到内核态

OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理



系统调用

系统调用的过程

用户代码中的read()只是一个库函数
并非真正的系统调用

库函数准备所有必要的参数
设置寄存器

read(fd,buf,count)

int 0x80/syscall

这个指令不会显式地出现在用户代码中
用户会以为是自己的read()引发了系统调用
库函数偷偷地执行一个软中断或陷阱指令
发起真正的系统调用

误区

“自陷/陷入”并不是指一条指令

让CPU陷入到内核态的是软中断指令/陷阱指令

而CPU陷入到内核态的这个动作属于自陷/陷入，它是一种现象

自陷/陷入这种异常指的是主动设置一起异常事件，让CPU进入异常处理
正好符合系统调用的思路

只有CPU进入内核态，才能执行OS的代码，应用才能请求OS的服务

模式位=0

PSW

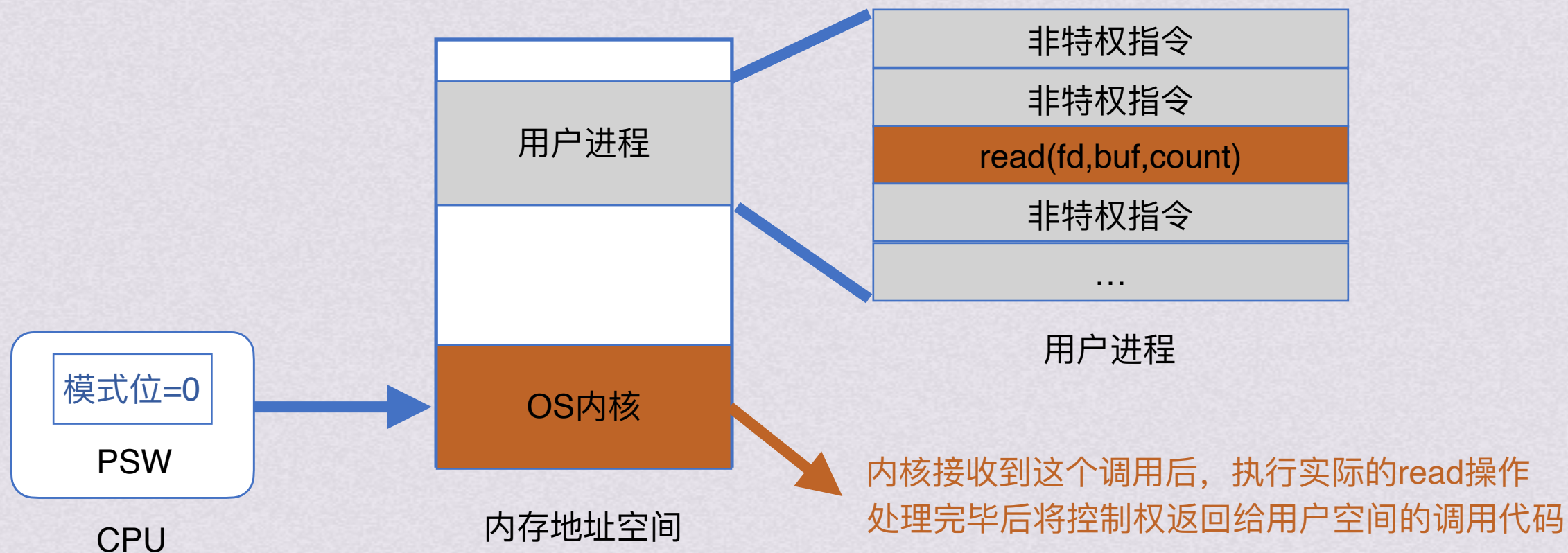
CPU

执行软中断/陷阱指令
可以引发自陷异常
导致CPU从用户态陷入到内核态

OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理



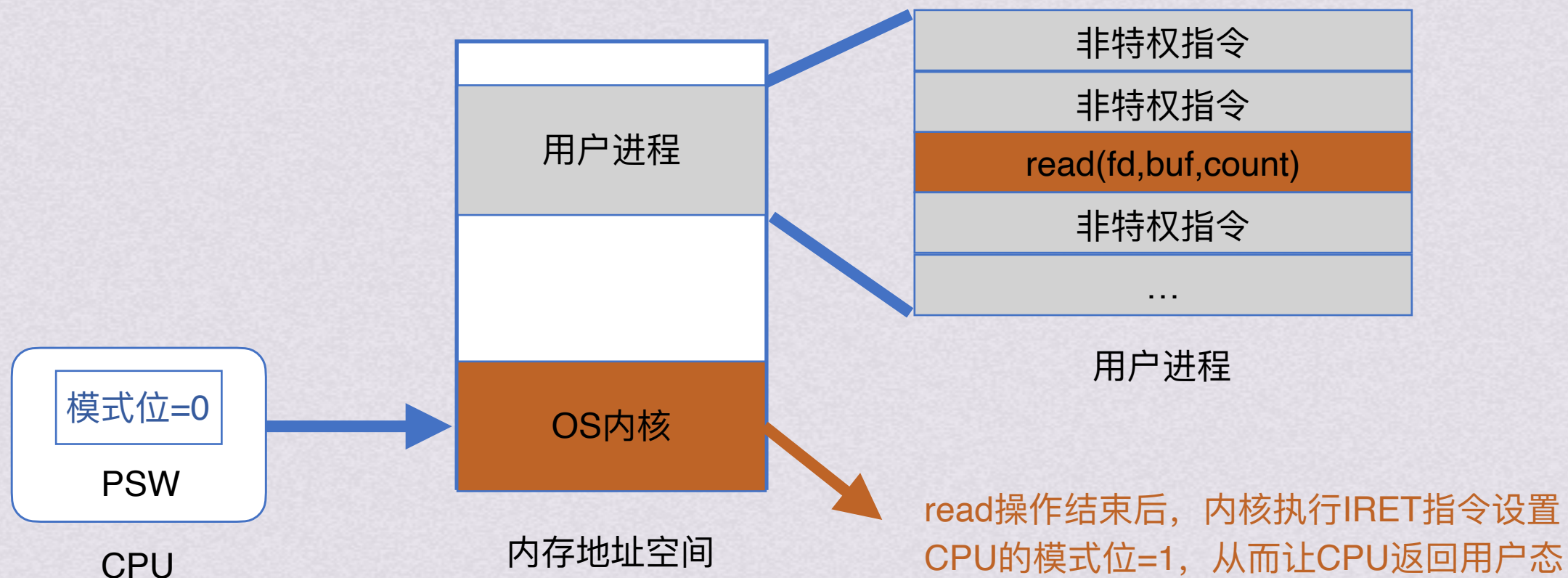
系统调用



OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理



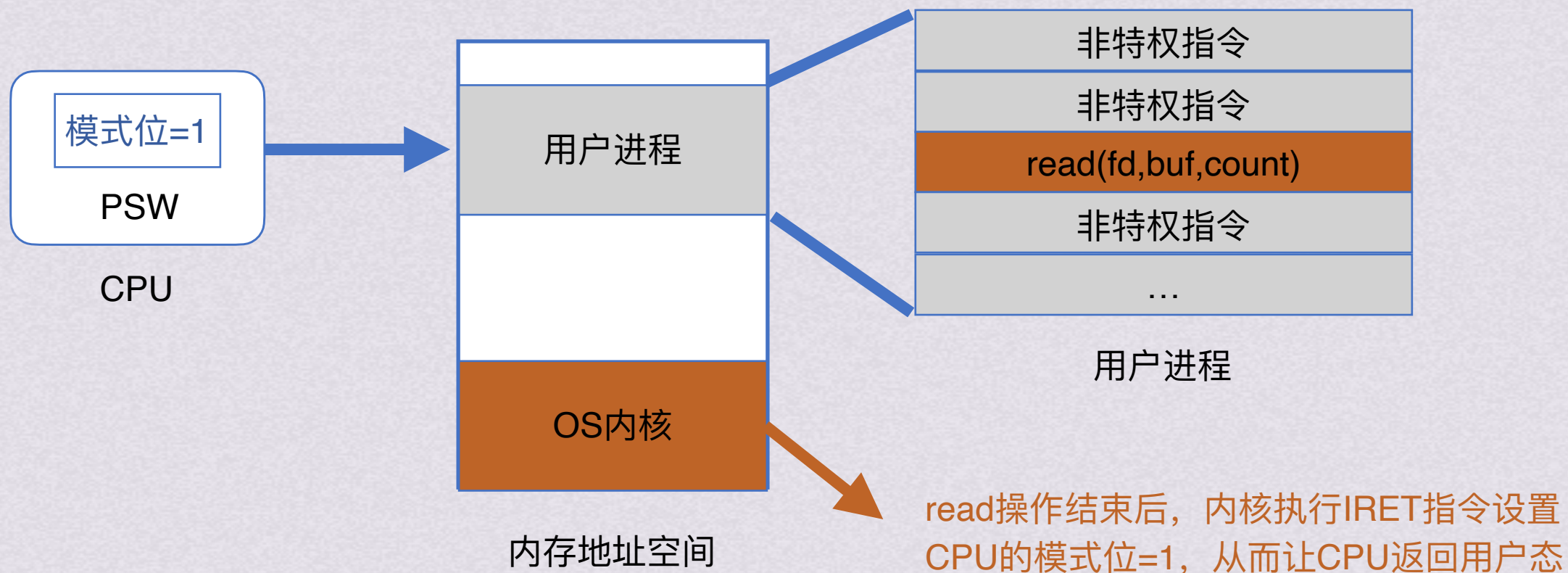
系统调用



OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理



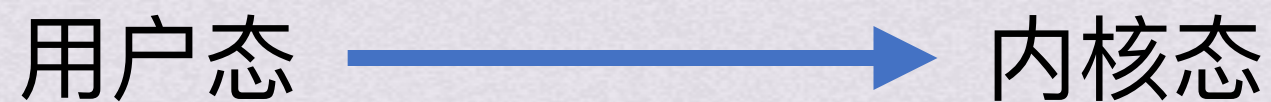
系统调用



OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理



系统调用



1. 用户程序发起系统调用
2. 发生中断，必须陷入内核态由OS内核处理中断
3. 发生异常，必须陷入内核态由OS内核处理中断
4. 用户程序试图执行特权指令

OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理



系统调用

用户态 ←————— 内核态

设置程序状态字寄存器PSW中的CPU模式位，将其置1

{ OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
中断和异常的处理

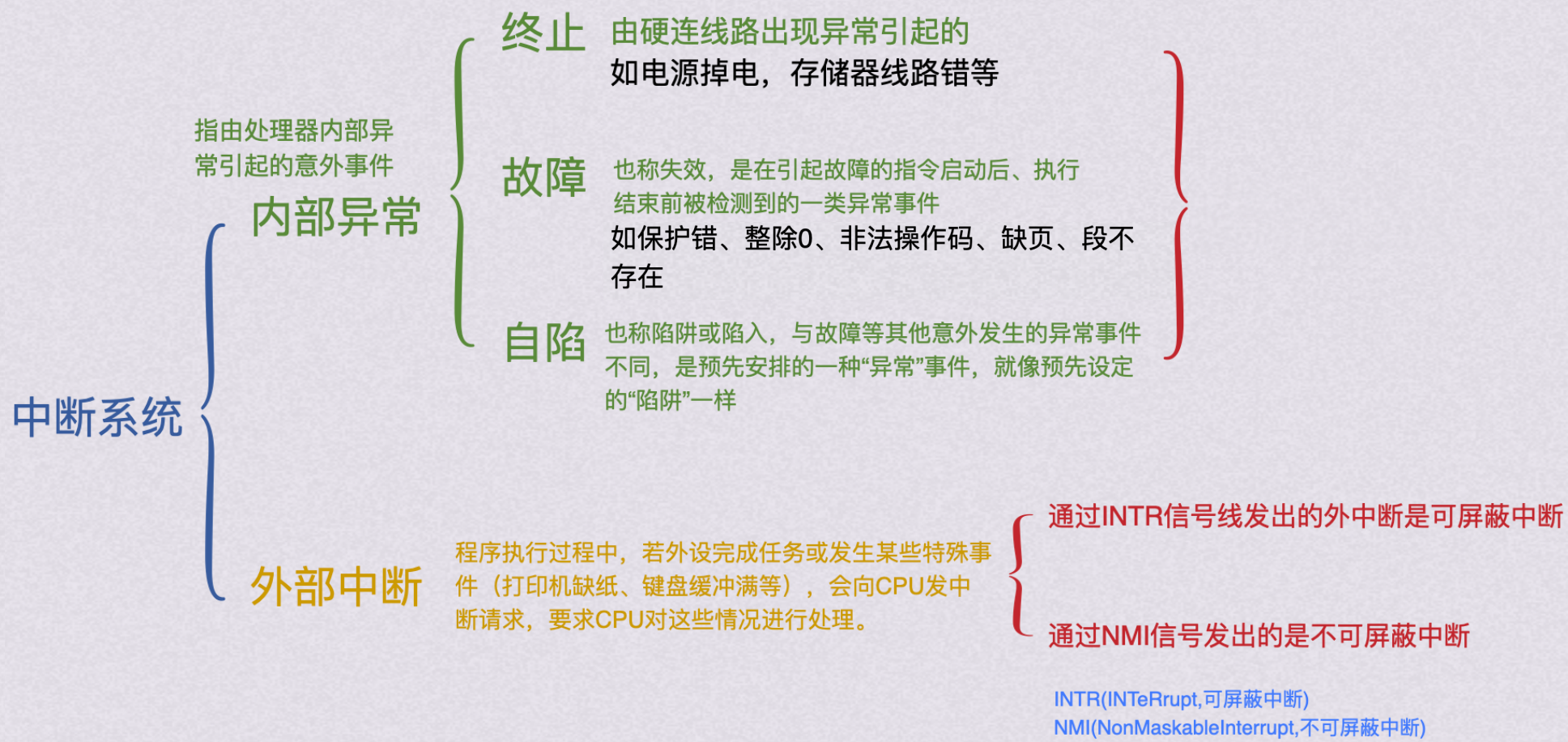


中断和异常的处理

{ OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用

中断和异常的处理

现代操作系统由中断驱动



- OS引导
- OS内核
- CPU的双重工作模式
- 特权指令&非特权指令
- 系统调用



中断和异常的处理

现代操作系统由中断驱动

异常处理过程

异常触发：当程序执行中发生异常，CPU会自动检测并触发异常

保存上下文：与中断处理相似，CPU会保存当前的执行状态，包括程序计数器和寄存器值

执行异常处理程序：CPU查找异常向量表，根据异常类型跳转到相应异常处理程序处理

处理结果：根据异常性质，处理程序可能会修正错误并允许程序继续执行，或终止程序返回错误信息

恢复或终止：如果异常可恢复，CPU将恢复之前的程序状态并继续执行；若不可恢复，终止程序并进行错误处理

OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用



中断和异常的处理

现代操作系统由中断驱动

中断处理过程

触发中断：中断指由外部设备（键盘、鼠标等）发起，通常是设备完成任务或需要注意时触发

中断信号接收：CPU会在当前执行的指令完成后检测中断，暂停当前进程的执行

保存上下文：CPU会保存当前任务的状态，包括程序计数器和其他寄存器值（现场信息），以便中断处理完成后能恢复运行

执行中断服务程序：CPU根据中断向量表找到相应中断处理程序并执行

恢复上下文并返回：中断服务程序完成后，CPU恢复之前保存的程序状态，继续执行被中断程序

OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用



中断和异常的处理

现代操作系统由中断驱动

异常处理过程

异常触发：当程序执行中发生异常，CPU会自动检测并触发异常

保存上下文：与中断处理相似，CPU会保存当前的执行状态，包括程序计数器和寄存器值

执行异常处理程序：CPU查找异常向量表，根据异常类型跳转到相应异常处理程序处理

处理结果：根据异常性质，处理程序可能会修正错误并允许程序继续执行，或终止程序返回错误信息

恢复或终止：如果异常可恢复，CPU将恢复之前的程序状态并继续执行；若不可恢复，终止程序并进行错误处理

OS引导
OS内核
CPU的双重工作模式
特权指令&非特权指令
系统调用

中断处理过程

触发中断：中断指由外部设备（键盘、鼠标等）发起，通常是设备完成任务或需要注意时触发

中断信号接收：CPU会在当前执行的指令完成后检测中断，暂停当前进程的执行

保存上下文：CPU会保存当前任务的状态，包括程序计数器和其他寄存器值（现场信息），以便中断处理完成后能恢复运行

执行中断服务程序：CPU根据中断向量表找到相应中断处理程序并执行

恢复上下文并返回：中断服务程序完成后，CPU恢复之前保存的程序状态，继续执行被中断程序